

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Лабораторная работа №3

Контейнеризация микросервисов (Docker)

по дисциплине «Программирование серверной логики»

Вариант: Сайт для поиска работы

Студент: Гельм Даниил

Группа: K3340 / БР1.1

Санкт-Петербург
2026

Содержание

1. Цель работы
2. Исходная система
3. Dockerfile для сервисов
4. Docker Compose
5. Сборка и запуск
6. Проверка работоспособности
7. Выводы

1. Цель работы

Цель — контейнеризовать микросервисную систему Job Search Platform (JP2 + RabbitMQ из Д35): написать Dockerfile для каждого сервиса и описать всю инфраструктуру в docker-compose.yml, чтобы система поднималась одной командой.

После выполнения работы не требуется локальный npm run dev: достаточно docker compose up для запуска PostgreSQL, RabbitMQ, всех микросервисов и API Gateway.

2. Исходная система

База — проект labs/lab2: API Gateway (порт 3000), 7 микросервисов с PostgreSQL (auth, company, skills, resume, vacancy, application, favorites), Notification Service (порт 3008, RabbitMQ consumer). Межсервисное взаимодействие: HTTP + RabbitMQ.

Компонент	Порт	Назначение
gateway	3000	Единая точка входа API
auth	3001	Регистрация, вход, JWT
company	3002	Компании работодателей
skills	3003	Справочник навыков
resume	3004	Резюме соискателей
vacancy	3005	Вакансии
application	3006	Отклики, публикация в RabbitMQ
favorites	3007	Избранные вакансии
notification	3008	Обработка событий RabbitMQ
postgres	5432 (5434 с хоста)	7 баз данных
rabbitmq	5672, 15672	Очередь сообщений

3. Dockerfile для сервисов

Для каждого сервиса в labs/lab2 создан multi-stage Dockerfile. Контекст сборки — корень lab2 (нужен каталог shared/). Этап build: npm ci, tsc. Этап runtime: node dist/.../index.js.

Сервис	Файл Dockerfile
Gateway	labs/lab2/gateway/Dockerfile
Auth	labs/lab2/services/auth/Dockerfile
Company	labs/lab2/services/company/Dockerfile
Skills	labs/lab2/services/skills/Dockerfile
Resume	labs/lab2/services/resume/Dockerfile
Vacancy	labs/lab2/services/vacancy/Dockerfile
Application	labs/lab2/services/application/Dockerfile
Favorites	labs/lab2/services/favorites/Dockerfile
Notification	labs/lab2/services/notification/Dockerfile

Пример структуры Dockerfile (auth):

```
FROM node:22-alpine AS build
COPY shared ./shared
COPY services/auth/ ...
RUN npm ci && npm run build

FROM node:22-alpine
CMD ["node", "dist/services/auth/src/index.js"]
```

Файл labs/lab2/.dockerignore исключает node_modules, dist, .env и PDF-отчёты из контекста сборки.

```
Dockerfile ×
BP1.1 > Gelm_Daniil > labs > lab2 > gateway > Dockerfile Keep all changes Review Next F
1 FROM node:22-alpine AS build
2 WORKDIR /app
3 COPY shared ./shared
4 COPY gateway/package*.json ./gateway/
5 RUN cd gateway && npm ci
6 COPY gateway ./gateway
7 WORKDIR /app/gateway
8 RUN npm run build
9
10 FROM node:22-alpine
11 WORKDIR /app/gateway
12 ENV NODE_ENV=production
13 COPY --from=build /app/gateway/dist ./dist
14 COPY --from=build /app/gateway/node_modules ./node_modules
15 COPY --from=build /app/gateway/package.json ./
16 EXPOSE 3000
17 CMD ["node", "dist/gateway/src/index.js"]
18
```

4. Docker Compose

labs/lab2/docker-compose.yml описывает 11 сервисов: postgres, rabbitmq и 9 Node-приложений. Переменные окружения для Docker-сети: DB_HOST=postgres, сервисные URL вида http://auth:3001, RABBITMQ_URL=amqp://guest:guest@rabbitmq:5672.

depends_on с condition: service_healthy для postgres и rabbitmq. Gateway стартует после всех upstream-микросервисов. Наружу проброшены порты: 3000 (API), 3008 (notification), 5434 (postgres), 5672

и 15672 (rabbitmq).

```
Problems  Output  Debug Console  Terminal  Ports

● daniilgelm@m1 ITMO-ACS-Backend-2026-A % cd BP1.1
● daniilgelm@m1 BP1.1 % cd Gelm_Daniil
● daniilgelm@m1 Gelm_Daniil % cd labs
● daniilgelm@m1 labs % cd lab2
● daniilgelm@m1 lab2 % docker compose ps
NAME                SERVICE          IMAGE              STATUS              COMMAND              PO
RTS
lab2-application-1   application       lab2-application    Up 9 minutes        "docker-entrypo      30
int.s..."          06/tcp
lab2-auth-1          auth             lab2-auth           Up 9 minutes        "docker-entrypo      30
int.s..."          01/tcp
lab2-company-1       company          lab2-company        Up 9 minutes        "docker-entrypo      30
int.s..."          02/tcp
lab2-favorites-1     favorites        lab2-favorites      Up 9 minutes        "docker-entrypo      30
int.s..."          07/tcp
lab2-gateway-1       gateway          lab2-gateway        Up 9 minutes        "docker-entrypo      0.
0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
lab2-notification-1 notification      lab2-notification    Up 9 minutes        "docker-entrypo      0.
0.0.0:3008->3008/tcp, [::]:3008->3008/tcp
lab2-postgres-1      postgres         postgres:16-alpine   Up 9 minutes (healthy) 0.
0.0.0:5434->5432/tcp, [::]:5434->5432/tcp
lab2-rabbitmq-1      rabbitmq         rabbitmq:3-management-alpine "docker-entrypo      0.
int.s..."          rabbitmq         9 minutes ago       Up 9 minutes (healthy) 0.
0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 0.0.0.0:15672->15672/tcp,
[::]:15672->15672/tcp
lab2-resume-1        resume          lab2-resume         Up 9 minutes        "docker-entrypo      30
int.s..."          04/tcp
lab2-skills-1        skills          lab2-skills         Up 9 minutes        "docker-entrypo      30
int.s..."          03/tcp
lab2-vacancy-1       vacancy         lab2-vacancy        Up 9 minutes        "docker-entrypo      30
int.s..."          05/tcp
○ daniilgelm@m1 lab2 %
```

5. Сборка и запуск

```
cd labs/lab2
npm run docker:up
# или: docker compose up -d --build

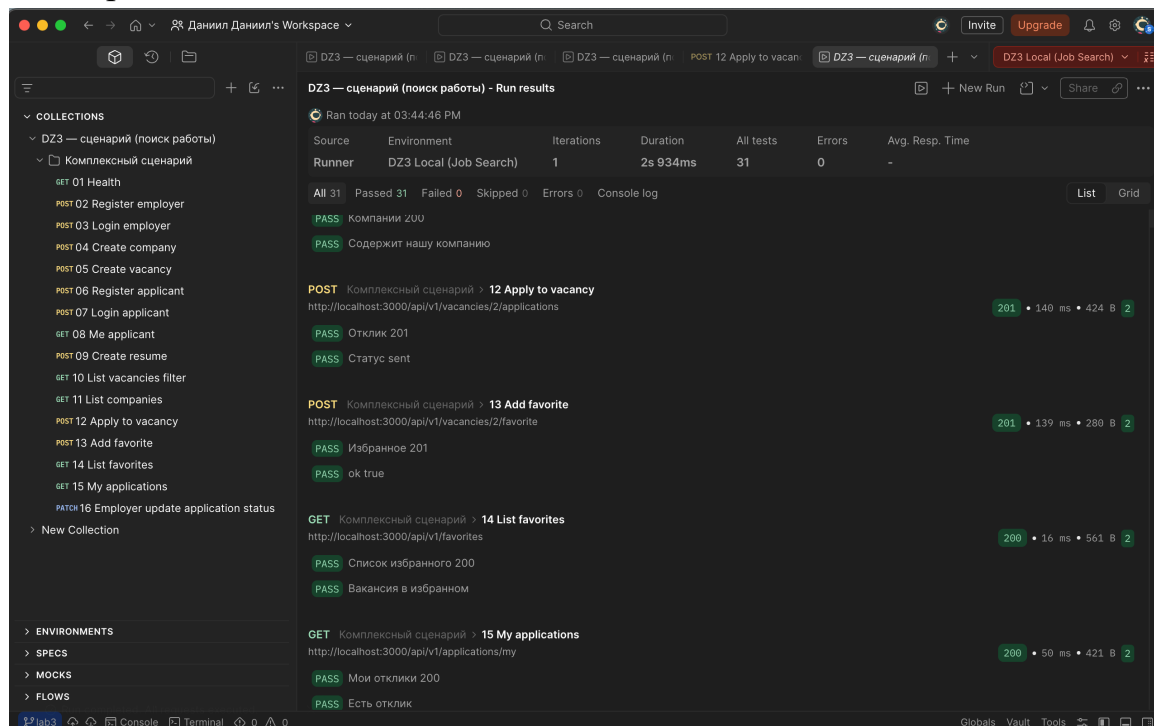
npm run docker:down # остановка
```

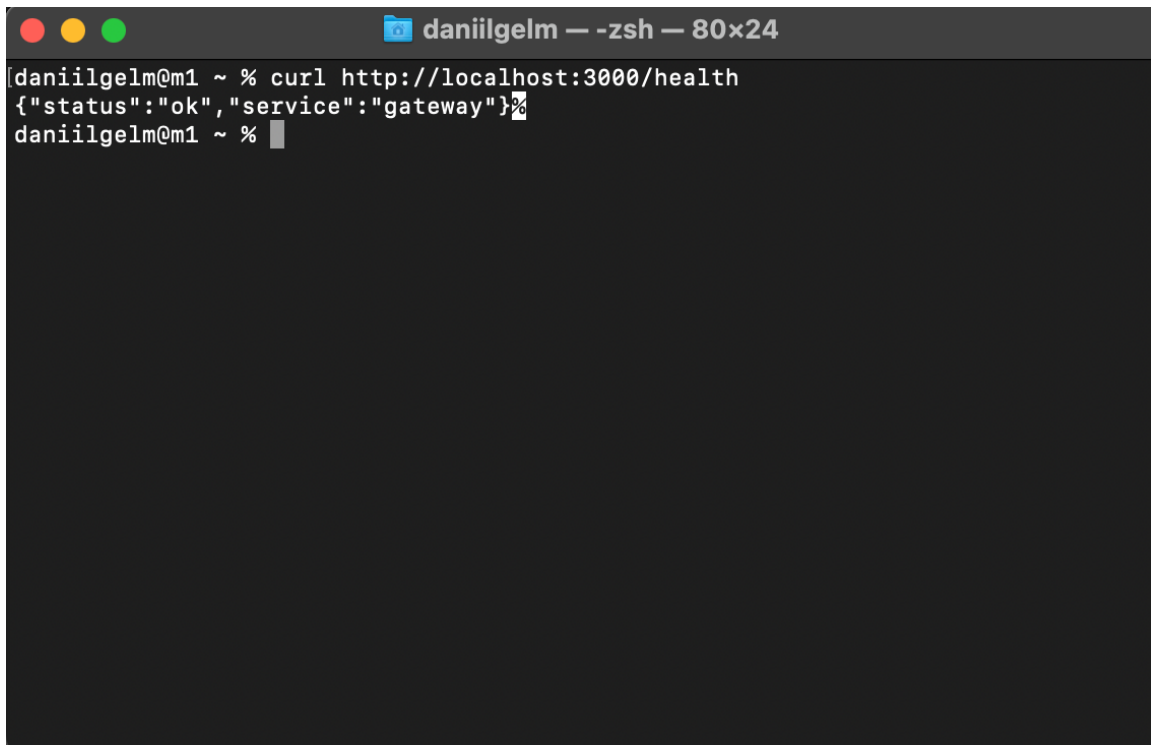
Команда `docker:up` собирает образы всех сервисов и поднимает стек. Первый запуск занимает несколько минут (скачивание `node:22-alpine`, `npm ci`, `tsc`).

6. Проверка работоспособности

Проверка health gateway: `GET http://localhost:3000/health` → `{"status":"ok"}`.

Комплексный Postman-сценарий из ДЗ3 (регистрация, вакансия, отклик, избранное, смена статуса) прогнан через `localhost:3000` — все 16 запросов и 31 assertion Passed.



A terminal window with a dark background and a title bar that reads "daniilgelm — zsh — 80x24". The terminal shows a user prompt "daniilgelm@m1 ~ %" followed by the command "curl http://localhost:3000/health". The output of the command is a JSON object: {"status":"ok","service":"gateway"}. The prompt then returns to "daniilgelm@m1 ~ %".

```
daniilgelm@m1 ~ % curl http://localhost:3000/health
{"status":"ok","service":"gateway"}%
daniilgelm@m1 ~ %
```

7. Выводы

Микросервисная система Job Search Platform полностью контейнеризована. Каждый сервис собирается в отдельный Docker-образ. Одна команда `docker compose up` поднимает PostgreSQL, RabbitMQ, все микросервисы и API Gateway. Функциональность API сохранена — Postman-сценарий выполняется успешно.